DISEÑO DE ALGORITMOS

ASESOR:
ING. GERARDO ACEVEDO VEGA

UNIDAD 3 · ACTIVIDAD COMPLEMENTARIA 1

OPERADORES, PRECEDENCIA Y EVALUACIÓN DE EXPRESIONES

16 DE OCTUBRE DE 2026

GRUPO 8191 · INFORMÁTICA

Alumno: Mauro Noel Sánchez Campos

Universidad Nacional Autónoma de México

Facultad de Contaduría y Administración

Introducción

Cuando la computadora evalúa una expresión como `7 + 3 * 2`, no la lee de izquierda a derecha como un texto. Sigue reglas: primero resuelve los operadores con mayor **precedencia** y, cuando dos operadores tienen la misma precedencia, decide el orden según su **asociatividad**, que puede ser de izquierda a derecha o de derecha a izquierda (Kanagusico et al., 2026). Si no se conocen estas reglas, es muy fácil escribir una expresión que da un resultado distinto al que uno esperaba. En esta actividad elaboro la tabla de precedencia de los operadores de C que se piden y evalúo paso a paso tres expresiones.

1. Tabla de precedencia y asociatividad

La tabla va de mayor a menor prioridad. Los operadores que están en el mismo nivel tienen la misma precedencia y se resuelven según la asociatividad de ese nivel.

Nivel	Operadores	Tipo	Asociatividad
1	`( )`	Agrupación (paréntesis)	Izquierda a derecha
2	`!` `-` (unario)	Lógico NOT y signo negativo	Derecha a izquierda
3	`* / %`	Aritméticos multiplicativos	Izquierda a derecha
4	`+ -`	Aritméticos aditivos	Izquierda a derecha
5	`< <= > >=`	Relacionales de orden	Izquierda a derecha
6	`== !=`	Relacionales de igualdad	Izquierda a derecha
7	`&&`	Lógico AND (Y)	Izquierda a derecha
8	`||`	Lógico OR (O)	Izquierda a derecha
9	`=`	Asignación	Derecha a izquierda

Tabla 1. Precedencia y asociatividad de operadores en C. Elaboración propia con base en Kernighan y Ritchie (1991) y Kanagusico et al. (2026).

Algunas observaciones que me ayudaron a entender la tabla:

- Los paréntesis siempre se resuelven primero, por eso son la forma más segura de forzar el orden que uno quiere.
- `%` (residuo) tiene la misma precedencia que `*` y `/`; no es un operador "menor".

- Los relacionales `<`, `>` van antes que los de igualdad `==`, `!=`.
- `&&` tiene mayor precedencia que `||`, igual que en matemáticas la multiplicación va antes que la suma.
- La asignación es de derecha a izquierda: en `a = b = 5` primero se hace `b = 5` y después `a = b`.

2. Evaluación paso a paso

a) 7 + 3 * 2 - 10 / 5

Paso	Operador que se resuelve	Justificación	Expresión resultante
1	`3 * 2 = 6`	`*` tiene mayor precedencia que + y `-`, y está más a la izquierda que `/`	`7 + 6 - 10 / 5`
2	`10 / 5 = 2`	`/` tiene la misma precedencia que `*`; se resuelve antes que + y `-`	`7 + 6 - 2`
3	`7 + 6 = 13`	+ y `-` tienen la misma precedencia; por asociatividad de izquierda a derecha va primero +	`13 - 2`
4	`13 - 2 = 11`	Último operador	`11`

> **Resultado final: 11 — tipo entero.** Todos los operandos son enteros y la división `10 / 5` es exacta, así que no aparecen decimales.

b) (8 > 5) && (3 == 3) || (4 < 2)

Paso	Operador que se resuelve	Justificación	Expresión resultante
1	`(8 > 5) = 1` (verdadero)	Los paréntesis tienen la máxima precedencia; se resuelven de izquierda a derecha	`1 && (3 == 3) || (4 < 2)`
2	`(3 == 3) = 1` (verdadero)	Segundo paréntesis, de izquierda a derecha	`1 && 1 || (4 < 2)`
3	`(4 < 2) = 0` (falso)	Tercer paréntesis	`1 && 1 || 0`
4	`1 && 1 = 1`	&& tiene mayor precedencia que `||`; verdadero Y verdadero es verdadero	`1 || 0`
5	`1 || 0 = 1`	Último operador; verdadero O falso es verdadero	`1`

> **Resultado final: 1 (verdadero) — tipo booleano.** En C no existe un tipo booleano nativo en el estándar clásico: los operadores relacionales y lógicos devuelven un `int` con valor 1 (verdadero) o 0 (falso). Un detalle: C usa evaluación en cortocircuito, así que al ver que `1 && 1` ya es verdadero, en la práctica ni siquiera necesita evaluar `(4 < 2)` para saber el resultado del `||`; el resultado es el mismo.

c) 15 % 4 + 2 * 3

Paso	Operador que se resuelve	Justificación	Expresión resultante
1	`15 % 4 = 3`	% y * tienen la misma precedencia y es mayor que la de +; por asociatividad de izquierda a derecha va primero %. 15 entre 4 es 3 y sobran 3	`3 + 2 * 3`
2	`2 * 3 = 6`	* tiene mayor precedencia que +	`3 + 6`
3	`3 + 6 = 9`	Último operador	`9`

> **Resultado final: 9 — tipo entero.** El operador % solo trabaja con enteros en C, así que toda la expresión es entera.

3. Comprobación en C

Para confirmar los resultados escribí un programa corto que imprime las tres expresiones:

```
#include <stdio.h>

int main(void) {
    printf("a) %d\n", 7 + 3 * 2 - 10 / 5);              /* 11 */
    printf("b) %d\n", (8 > 5) && (3 == 3) || (4 < 2)); /* 1  */
    printf("c) %d\n", 15 % 4 + 2 * 3);                  /* 9  */
    return 0;
}
```

La salida coincide con la evaluación manual: `a) 11`, `b) 1` y `c) 9`.

Conclusión

La precedencia funciona como las reglas de jerarquía de operaciones que vimos en matemáticas, pero en C hay más niveles porque se agregan los operadores relacionales, lógicos y de asignación. Lo que más me sirvió fue la expresión b): si no supiera que `&&` va antes que `||`, podría resolverla en otro orden y, con otros valores, obtener un resultado distinto. Por eso, aunque el compilador sepa la precedencia, me parece buena práctica usar paréntesis cuando una expresión mezcla varios tipos de operadores: hace el código más fácil de leer y evita errores.

Referencias

Kanagusico Hernández, E. D., Moreno García, R. C., y Hernández Sánchez, E. (2026). *Diseño de algoritmos* [Apunte digital, Licenciatura en Informática]. Universidad Nacional Autónoma de México, Facultad de Contaduría y Administración.

Kernighan, B. W., y Ritchie, D. M. (1991). *El lenguaje de programación C* (2a ed.). Prentice Hall Hispanoamericana.